hw2_22000086_HoangXuanDuc

Để tìm cận trên (Upper Bound) theo ký hiệu Big-O (O), chúng ta cần tìm một hằng số dương c và một giá trị n_0 sao cho $f(n) \leq c \cdot g(n)$ với mọi $n \geq n_0$.

Dưới đây là lời giải chi tiết cho từng hàm số:

a) $f(n) = 4n + 5$

- **Phân tích:** Ta có $4n + 5 \leq 4n + 5n = 9n$ với mọi $n \geq 1$.
- **Kết luận:** Chọn $c = 9$ và $n_0 = 1$.
- **Cận trên:**
$$f(n) = O(n)$$

b) $f(n) = n^2 + 2$

- **Phân tích:** Ta có $n^2 + 2 \leq n^2 + 2n^2 = 3n^2$ với mọi $n \geq 1$.
- **Kết luận:** Chọn $c = 3$ và $n_0 = 1$.
- **Cận trên:**
$$f(n) = O(n^2)$$

c) $f(n) = n^4 + 100n^2 + 50$

- **Phân tích:** Với mọi $n \geq 1$, ta có $n^2 \leq n^4$ và $1 \leq n^4$. Do đó: $n^4 + 100n^2 + 50 \leq n^4 + 100n^4 + 50n^4 = 151n^4$.
- **Kết luận:** Chọn $c = 151$ và $n_0 = 1$.
- **Cận trên:**
$$f(n) = O(n^4)$$

d) $f(n) = 2n^3 - 2n^2$

- **Phân tích:** Vì $2n^2 \geq 0$ với mọi $n \geq 1$, nên khi trừ đi một số dương, giá trị sẽ nhỏ hơn số bị trừ. Ta có: $2n^3 - 2n^2 \leq 2n^3$ với mọi $n \geq 1$.
- **Kết luận:** Chọn $c = 2$ và $n_0 = 1$.
- **Cận trên:**
$$f(n) = O(n^3)$$

e) $f(n) = 508$

- **Phân tích:** Đây là một hàm hằng số. Ta có $508 \leq 508 \cdot 1$ với mọi $n \geq 1$.
- **Kết luận:** Chọn $c = 508$ và $n_0 = 1$.
- **Cận trên:**
$$f(n) = O(1)$$

Phân tích độ phức tạp thuật toán (Big-O)

Example 2

Đề bài: Tính $S = \frac{n*(n-1)}{2}$.

- **Giải thích:** Phép tính này chỉ bao gồm các thao tác toán học cơ bản (nhân, trừ, chia) và được thực hiện tuần tự. Không có vòng lặp hay đệ quy nào phụ thuộc vào kích thước của n. Do đó, thời gian thực thi là một hằng số.
- **Độ phức tạp thời gian:**
$$O(1)$$

Example 3

[language=Python] for i in range(0, n): print('Current Number', i)

- **Giải thích:** Hàm chứa một vòng lặp `for` chạy biến `i` từ 0 đến $n - 1$. Tổng cộng vòng lặp này thực thi n lần. Lệnh `print` bên trong tốn thời gian hằng số $O(1)$. Vậy thời gian chạy tỷ lệ thuận với n.
- **Độ phức tạp thời gian:**
$$O(n)$$

Example 4

[language=Python] for i in range(0, n): print('Current Number', i) break

- **Giải thích:** Mặc dù khai báo vòng lặp `for` lặp n lần, nhưng lệnh `break` ngay lập tức kết thúc vòng lặp sau lần lặp đầu tiên. Do đó, đoạn code chỉ thực thi đúng 1 lần duy nhất bất kể giá trị đầu vào của n là bao nhiêu.
- **Độ phức tạp thời gian:**
$$O(1)$$

Example 5

[language=Python] def function(n): i = 1 while i <= n: i = i * 2 print(i) function(100)

- **Giải thích:** Vòng lặp `while` chạy chừng nào $i \leq n$. Sau mỗi vòng lặp, i được nhân đôi ($i = 1, 2, 4, 8, \dots$). Số lần lặp sẽ là giá trị k sao cho $2^k \approx n$, suy ra $k = \log_2(n)$.
- **Độ phức tạp thời gian:**
$$O(\log n)$$

Example 6

[language=Python] for i in range(0, n): for j in range(0, n): print("Value of i, j", i, j)

- **Giải thích:** Đoạn mã có 2 vòng lặp lồng nhau. Vòng lặp ngoài chạy n lần. Với mỗi lần vòng lặp ngoài chạy, vòng lặp trong cũng chạy n lần. Tổng số lần thực thi của lệnh `print` bên trong là $n \times n = n^2$.
- **Độ phức tạp thời gian:**
$$O(n^2)$$

Example 7

[language=Java] public void function(int n) int i, j, k, count = 0; for(i = n/2; i <= n; i++) for(j = 1; j + n/2 <= n; j++) for(k = 1; k <= n; k = k * 2) count++;

- **Giải thích:** Phân tích từng vòng lặp:

- Vòng lặp ngoài cùng (`i`): chạy từ $n/2$ đến n, bước nhảy là 1, do đó lặp khoảng $\frac{n}{2}$ lần $\Rightarrow O(n)$.
- Vòng lặp giữa (`j`): điều kiện `j + n/2 <= n` tương đương với `j <= n/2`. Nó chạy từ 1 đến $n/2$, bước nhảy 1, do đó lặp $\frac{n}{2}$ lần $\Rightarrow O(n)$.
- Vòng lặp trong cùng (`k`): chạy từ 1 đến n, mỗi lần k nhân đôi ($k = k * 2$), mất khoảng $\log_2(n)$ lần lặp $\Rightarrow O(\log n)$.

Vì ba vòng lặp này lồng nhau, số phép toán tổng cộng tỷ lệ với $(n/2) \cdot (n/2) \cdot \log_2(n) = \frac{n^2}{4} \log_2(n)$. Khi bỏ đi các hằng số, ta lấy phần tăng trưởng bậc cao nhất.

- **Độ phức tạp thời gian:**

$$O(n^2 \log n)$$

Example 8

[language=Java] public void function(int n) int i, j, k, count = 0; for(i = n/2; i <= n; i++) for(j = 1; j <= n; j = 2 * j) for(k = 1; k <= n; k = k * 2) count++;

- **Giải thích:** Phân tích từng vòng lặp:
  - Vòng lặp ngoài cùng (`i`): chạy từ $n/2$ đến n, bước nhảy là 1, lặp khoảng $\frac{n}{2}$ lần $\Rightarrow O(n)$.
  - Vòng lặp giữa (`j`): chạy từ 1 đến n, mỗi lần j nhân đôi ($j = 2 * j$), số lần lặp xấp xỉ $\log_2(n)$ lần $\Rightarrow O(\log n)$.
  - Vòng lặp trong cùng (`k`): chạy từ 1 đến n, mỗi lần k nhân đôi ($k = k * 2$), số lần lặp xấp xỉ $\log_2(n)$ lần $\Rightarrow O(\log n)$.

  Vì ba vòng lặp này lồng nhau, tổng số phép toán sẽ tỷ lệ với $\frac{n}{2} \cdot \log_2(n) \cdot \log_2(n)$. Lấy bậc cao nhất và loại bỏ các hằng số.

- **Độ phức tạp thời gian:**

$$O(n \log^2 n)$$

Example 9. What is the complexity of the algorithm?

Để tìm độ phức tạp tổng quát, ta giữ lại hạng tử có tốc độ tăng trưởng nhanh nhất khi n tiến tới vô cùng và bỏ qua các hằng số.

- **a)** $T(n) = n\log n + 3n + 2$
  - **Giải thích:** So sánh $n\log n$ và $3n$, ta thấy $n\log n$ lớn hơn khi n đủ lớn. Hạng tử chi phối là $n\log n$.
  - **Kết quả:**
$$O(n\log n)$$
- **b)** $T(n) = n\log(n!) + 5n^2 + 6$
  - **Giải thích:** Theo công thức xấp xỉ Stirling, $\log(n!) = O(n\log n)$. Do đó, $n\log(n!) = O(n \cdot n\log n) = O(n^2\log n)$. So sánh $n^2\log n$ và $5n^2$, rõ ràng $n^2\log n$ có bậc cao hơn và tăng trưởng nhanh hơn.
  - **Kết quả:**
$$O(n^2\log n)$$
- **c)** $T(n) = 100n + 10n^2$
  - **Giải thích:** Hạng tử có bậc cao nhất ở đây là $10n^2$. Ta bỏ qua hằng số 10.
  - **Kết quả:**
$$O(n^2)$$
- **d)** $T(n) = 50n\log n + n^3 + 100n$
  - **Giải thích:** Trong ba hạng tử: $n\log n$, n^3 và n, đa thức n^3 có bậc cao nhất và chi phối toàn bộ hàm.
  - **Kết quả:**
$$O(n^3)$$
- **e)** $T(n) = 10n\log n + n(\log n)^2$
  - **Giải thích:** Hạng tử $n(\log n)^2$ (có thể viết là $n\log^2 n$) chứa bình phương của log, do đó có tốc độ tăng trưởng nhanh hơn so với hạng tử $10n\log n$.
  - **Kết quả:**
$$O(n\log^2 n)$$

Example 10. What is the complexity of the algorithms in the code segments below?

a)

[language=Java] // Returns the sum of the integers in given array. public static int example1(int[] arr) int n = arr.length, total = 0; for (int j=0; j < n; j++) // loop from 0 to n-1 total += arr[j]; return total;

- **Giải thích:** Thuật toán có một vòng lặp `for` duy nhất, lặp từ 0 đến $n-1$. Các phép gán và cộng bên trong lặp tốn thời gian hằng số $O(1)$. Do đó, thời gian chạy tỷ lệ thuận với kích thước mảng n.
- **Độ phức tạp thời gian:**

$$O(n)$$

b)

[language=Java] // Returns the sum of the integers with even index in given array. public static int example2(int[] arr) int n = arr.length, total = 0; for (int j=0; j < n; j += 2) // note the increment of 2 total += arr[j]; return total;

- **Giải thích:** Vòng lặp `for` chạy từ 0 đến $n-1$ nhưng với bước nhảy là 2 (`j += 2`). Tổng số lần lặp sẽ là $\frac{n}{2}$. Khi bỏ qua hằng số $\frac{1}{2}$, tốc độ tăng trưởng vẫn là tuyến tính theo n.
- **Độ phức tạp thời gian:**

$$O(n)$$

c)

[language=Java] // Returns the sum of the prefix sums of given array. public static int example3(int[] arr) int n = arr.length, total = 0; for (int j=0; j < n; j++) // loop from 0 to n-1 for (int k=0; k <= j; k++) // loop from 0 to j total += arr[j]; return total;

- **Giải thích:** Thuật toán gồm 2 vòng lặp lồng nhau. Vòng ngoài chạy n lần (từ $j = 0$ đến $n-1$). Với mỗi giá trị của j, vòng trong chạy $j+1$ lần (từ $k = 0$ đến j). Tổng số lần thực thi lệnh cộng là: $1+2+3+\cdots+n = \frac{n(n+1)}{2}$. Bậc cao nhất của đa thức này là n^2.
- **Độ phức tạp thời gian:**

$$O(n^2)$$

d)

[language=Java] // Returns the sum of the prefix sums of given array. public static int example4(int[] arr) int n = arr.length, prefix = 0, total = 0; for (int j=0; j < n; j++) // loop from 0 to n-1 prefix += arr[j]; total += prefix; return total;

- **Giải thích:** Tương tự như câu a, thuật toán này chỉ sử dụng một vòng lặp `for` lặp n lần. Dù bên trong vòng lặp có thực hiện hai phép cộng thay vì một, thời gian chạy thực thi của các khối lệnh bên trong vẫn là $O(1)$. Do đó, toàn bộ vòng lặp mất thời gian $O(n)$. Đây là một cách tối ưu thuật toán từ bài c (từ $O(n^2)$ xuống $O(n)$).

- **Độ phức tạp thời gian:**
$$O(n)$$

e)

[language=Java] // Returns the number of times second array stores sum of prefix sums from first. public static int example5(int[] first, int[] second) // assume equal-length arrays int n = first.length, count = 0; for (int i=0; i < n; i++) // loop from 0 to n-1 int total = 0; for (int j=0; j < n; j++) // loop from 0 to n-1 for (int k=0; k <= j; k++) // loop from 0 to j total += first[k]; if (second[i] == total) count++; return count;

- **Giải thích:** Thuật toán có 3 vòng lặp lồng nhau. Vòng lặp ngoài cùng (`i`) chạy n lần. Bên trong vòng `i`, có hai vòng lặp lồng nhau (`j` và `k`). Như đã phân tích ở câu c, hai vòng lặp `j` và `k` này mất $\frac{n(n+1)}{2}$ bước (tức là $O(n^2)$). Vì hai vòng lặp này thực thi trọn vẹn ở mỗi bước lặp của vòng `i`, tổng số phép toán sẽ là $n \cdot O(n^2)$.

- **Độ phức tạp thời gian:**
$$O(n^3)$$

Exercises 2.1

Bài 1

- **a. Tính trung bình của n số**
  - (i) Kích thước đầu vào: n (số lượng phần tử).
  - (ii) Phép toán cơ bản: Phép cộng.

- (iii) Sự thay đổi: Không. Luôn cần $n-1$ (hoặc n) phép cộng bất kể giá trị đầu vào.

- **b. Tính $n!$**
  - (i) Kích thước đầu vào: Giá trị của n.
  - (ii) Phép toán cơ bản: Phép nhân.
  - (iii) Sự thay đổi: Không. Luôn cần $n-1$ phép nhân.

- **c. Tìm phần tử nhỏ nhất trong danh sách n số**
  - (i) Kích thước đầu vào: n (số lượng phần tử).
  - (ii) Phép toán cơ bản: Phép so sánh.
  - (iii) Sự thay đổi: Không. Luôn cần duyệt qua danh sách và thực hiện $n-1$ phép so sánh.

- **d. Hiển thị đảo ngược danh sách n số**
  - (i) Kích thước đầu vào: n (số lượng phần tử).
  - (ii) Phép toán cơ bản: Phép in / hiển thị ra màn hình.
  - (iii) Sự thay đổi: Không. Luôn in ra đúng n lần.

- **e. Đảo ngược danh sách n số**
  - (i) Kích thước đầu vào: n (số lượng phần tử).
  - (ii) Phép toán cơ bản: Phép hoán vị (Swap) hoặc phép gán.
  - (iii) Sự thay đổi: Không. Số lần hoán vị luôn là $\lfloor n/2 \rfloor$.

- **f. Thuật toán giấy bút để cộng hai số nguyên thập phân n chữ số**
  - (i) Kích thước đầu vào: n (số lượng chữ số).
  - (ii) Phép toán cơ bản: Phép cộng hai chữ số (bao gồm cả cờ nhớ).
  - (iii) Sự thay đổi: Không. Luôn thực hiện thao tác cộng từ hàng đơn vị lên hàng cao nhất đúng n lần.

Bài 2

- **a. Thuật toán tìm hiệu giữa hai ma trận $n \times n$**
  - **Phép toán cơ bản:** Phép trừ giữa hai phần tử tương ứng.

- **Hàm theo bậc ma trận n:** Mỗi ma trận có n^2 phần tử, thuật toán duyệt qua từng vị trí nên cần thực hiện đúng n^2 phép trừ. Do đó:
$$C(n) = n^2$$
- **Hàm theo tổng số phần tử N:** Tổng số phần tử của 2 ma trận là $N = 2n^2$, suy ra $n^2 = \frac{N}{2}$. Vậy số phép toán theo N là:
$$C(N) = \frac{N}{2}$$

- **b. Thuật toán nhân ma trận $n \times n$ theo định nghĩa**
  - **Phép toán cơ bản:** Phép nhân (đây là thao tác chi phối thời gian thực thi).
  - **Hàm theo bậc ma trận n:** Ma trận kết quả có n^2 phần tử. Mỗi phần tử là tích vô hướng của một hàng và một cột (đều có độ dài n), cần n phép nhân. Tổng cộng:
$$C(n) = n^2 \cdot n = n^3$$
  - **Hàm theo tổng số phần tử N:** Tương tự, $N = 2n^2 \Rightarrow n = \sqrt{\frac{N}{2}}$. Số phép toán theo N là:
$$C(N) = \left(\sqrt{\frac{N}{2}}\right)^3 = \frac{N\sqrt{N}}{2\sqrt{2}}$$

Bài 3

Phân tích hiệu suất thuật toán **tìm kiếm tuần tự (Sequential search)**. Phép toán cơ bản ở đây là **phép so sánh khóa**.

- **Trường hợp xấu nhất (Worst case):**
  - **Giải thích:** Khóa cần tìm không có trong danh sách hoặc nằm ở vị trí cuối cùng. Thuật toán phải duyệt qua toàn bộ phần tử.
  - **Số phép so sánh:**
$$C_{worst}(n) = n$$
- **Trường hợp tốt nhất (Best case):**

- **Giải thích:** Khóa cần tìm nằm ngay ở vị trí đầu tiên. Thuật toán tìm thấy ngay lần kiểm tra đầu và dừng lại.
- **Số phép so sánh:**

$$C_{best}(n) = 1$$

- **Trường hợp trung bình (Average case):**
  - **Giải thích:** Giả sử xác suất khóa có trong danh sách là p ($0 \leq p \leq 1$). Nếu có mặt, xác suất nó nằm ở bất kỳ vị trí nào là $\frac{1}{n}$.
  - Số phép so sánh nếu tìm thấy (xác suất p): $\sum_{i=1}^{n} i \cdot \frac{1}{n} = \frac{n(n+1)}{2n} = \frac{n+1}{2}$
  - Số phép so sánh nếu không tìm thấy (xác suất $1-p$): n
  - **Tổng số phép so sánh trung bình:**

$$C_{avg}(n) = p\left(\frac{n+1}{2}\right) + (1-p)n$$

Bài 4

- **a. Lựa chọn găng tay (Glove selection)**
  - **Giải thích:** Có tổng cộng 22 chiếc găng tay (5 đôi đỏ = 10 chiếc, 4 đôi vàng = 8 chiếc, 2 đôi xanh = 4 chiếc). Mỗi đôi gồm 1 găng tay trái và 1 găng tay phải. Để tạo thành một đôi khớp nhau, ta cần 1 chiếc trái và 1 chiếc phải có cùng màu.
  - **Trường hợp tốt nhất (Best case):** Tình cờ bốc được ngay một đôi khớp màu ở 2 lần chọn đầu tiên. Số găng tay ít nhất cần lấy là **2**.
  - **Trường hợp xấu nhất (Worst case):** Trường hợp đen đủi nhất là ta bốc toàn bộ găng tay của cùng một bên (ví dụ: bốc tất cả 11 chiếc găng tay trái gồm 5 đỏ, 4 vàng, 2 xanh). Lúc này ta vẫn chưa có đôi nào khớp nhau. Chiếc găng tay thứ 12 được bốc ra chắc chắn sẽ là găng tay phải của một trong 3 màu đó và sẽ tạo thành một đôi. Vậy số găng tay ít nhất cần lấy để chắc chắn có 1 đôi là **12**.
- **b. Những chiếc tất bị thiếu (Missing socks)**
  - **Giải thích chung:** Có 5 đôi tất phân biệt (tổng cộng 10 chiếc). Mất 2 chiếc. Tổng số trường hợp xảy ra khi mất 2 chiếc tất bất kỳ là $\binom{10}{2} = 45$ cách.

- **Xác suất của trường hợp tốt nhất (Best-case scenario):** Trường hợp tốt nhất là mất 2 chiếc tất của *cùng một đôi*. Khi đó, bạn còn lại 4 đôi hoàn chỉnh. Có 5 đôi tất nên số cách để mất cả 2 chiếc của 1 đôi là 5 cách.

$$P_{best} = \frac{5}{45} = \frac{1}{9}$$

- **Xác suất của trường hợp xấu nhất (Worst-case scenario):** Trường hợp xấu nhất là mất 2 chiếc tất thuộc *hai đôi khác nhau*. Khi đó, bạn chỉ còn lại 3 đôi hoàn chỉnh. Số cách chọn 2 đôi bị ảnh hưởng là $\binom{5}{2} = 10$. Với mỗi đôi bị ảnh hưởng, có 2 cách mất 1 chiếc tất. Suy ra có $10 \cdot 2 \cdot 2 = 40$ cách.

$$P_{worst} = \frac{40}{45} = \frac{8}{9}$$

- **Số lượng đôi tất hoàn chỉnh dự kiến trong trường hợp trung bình (Average case):** Ta tính giá trị kỳ vọng (Expected value).

$$E = 4 \cdot P_{best} + 3 \cdot P_{worst} = 4 \cdot \frac{1}{9} + 3 \cdot \frac{8}{9} = \frac{28}{9} \approx 3.11 \text{ đôi}$$

Bài 5

- **a. Chứng minh công thức tính số bit** $b = \lfloor \log_2 n \rfloor + 1$
  - Một số nguyên dương n khi biểu diễn dưới dạng nhị phân cần b bit thì giá trị của nó sẽ nằm trong khoảng từ 2^{b-1} (số nhỏ nhất có b bit) đến $2^b - 1$ (số lớn nhất có b bit).
  - Ta có bất đẳng thức: $2^{b-1} \leq n < 2^b$.
  - Lấy logarit cơ số 2 các vế: $b - 1 \leq \log_2 n < b$.
  - Theo định nghĩa của hàm phần nguyên dưới (floor function), số nguyên lớn nhất bé hơn hoặc bằng $\log_2 n$ chính là $b - 1$. Do đó: $\lfloor \log_2 n \rfloor = b - 1 \implies b = \lfloor \log_2 n \rfloor + 1$ (ĐPCM).
- **b. Chứng minh công thức thay thế** $b = \lceil \log_2(n+1) \rceil$
  - Từ bất đẳng thức $2^{b-1} \leq n < 2^b$, vì n là số nguyên nên $n \leq 2^b - 1 \implies n + 1 \leq 2^b$.
  - Kết hợp lại, ta có: $2^{b-1} < n + 1 \leq 2^b$.

- Lấy logarit cơ số 2: $b - 1 < \log_2(n+1) \leq b$.
- Theo định nghĩa hàm phần nguyên trên (ceiling function), số nguyên nhỏ nhất lớn hơn hoặc bằng $\log_2(n+1)$ chính là b. Do đó: $b = \lceil \log_2(n+1) \rceil$ (ĐPCM).

- **c. Công thức tương tự cho số lượng chữ số thập phân**
  - Tương tự như hệ nhị phân (cơ số 2), hệ thập phân sử dụng cơ số 10. Nếu d là số lượng chữ số thập phân của số n, ta có các công thức:
  $$d = \lfloor \log_{10} n \rfloor + 1 \quad \text{hoặc} \quad d = \lceil \log_{10}(n+1) \rceil$$

- **d. Tại sao dùng hệ nhị phân hay hệ thập phân để đo kích thước n không quan trọng trong phân tích thuật toán?**
  - **Giải thích:** Theo công thức đổi cơ số của logarit: $\log_2 n = \log_2 10 \cdot \log_{10} n$.
  - Hai hàm số này chỉ khác nhau ở một hằng số nhân (là $\log_2 10 \approx 3.32$). Trong khuôn khổ phân tích độ phức tạp thuật toán (sử dụng ký hiệu Big-O, Θ, Ω), các hằng số nhân đều được bỏ qua. Do đó, $O(\log_2 n)$ và $O(\log_{10} n)$ đều được viết chung là $O(\log n)$. Sự khác biệt về cơ số đếm không làm thay đổi bậc phát triển (order of growth) của thuật toán.

Bài 6

- **Cách cải tiến thuật toán sắp xếp để best-case bằng $n-1$ phép so sánh:**
  - **Đề xuất:** Thêm một bước "tiền xử lý"(preprocessing step) trước khi chạy thuật toán sắp xếp chính. Cụ thể, ta duyệt qua mảng một lượt và so sánh từng cặp phần tử liền kề (so sánh $A[i]$ và $A[i+1]$ với i từ 1 đến $n-1$). Bước này tiêu tốn đúng $n-1$ phép so sánh.
  - Nếu phát hiện mảng đã được sắp xếp tăng dần, thuật toán lập tức dừng lại. Ngược lại, tiếp tục chạy thuật toán sắp xếp nguyên bản. Nhờ đó, trong trường hợp tốt nhất (best-case) khi mảng đã được sắp xếp sẵn, số phép so sánh chỉ là $n-1$.

- **Sự bổ sung này có đáng giá không?**
  - **Trả lời:** Nhìn chung là **Không đáng giá (Not worthwhile)**.

- **Lý do:** Nếu mảng đầu vào ngẫu nhiên, xác suất mảng đã được sắp xếp sẵn là cực kỳ nhỏ. Việc thêm bước kiểm tra này sẽ cộng thêm $n-1$ phép so sánh thừa thãi vào thời gian chạy của *mọi* trường hợp khác, làm tăng hệ số hằng số (overhead) của thuật toán. Nó chỉ thực sự có ích nếu chúng ta biết trước dữ liệu có tính chất "gần như đã sắp xếp"(nearly sorted). Bên cạnh đó, các thuật toán như Insertion Sort hay Bubble Sort (có cờ tối ưu) vốn dĩ đã có độ phức tạp best-case là $O(n)$ mà không cần phải gượng ép thêm bước kiểm tra riêng rẽ này.

Bài 7

- **Giải thích chung:** Thuật toán khử Gauss có số lượng phép nhân xấp xỉ $\frac{1}{3}n^3$. Suy ra thời gian thực thi $T(n)$ tỉ lệ thuận với n^3.
- **a. Thời gian giải hệ 1000 phương trình so với 500 phương trình**
  - Ta lập tỉ số thời gian chạy giữa kích thước $n = 1000$ và $n = 500$:

$$\frac{T(1000)}{T(500)} \approx \frac{\frac{1}{3} \cdot 1000^3}{\frac{1}{3} \cdot 500^3} = \left(\frac{1000}{500}\right)^3 = 2^3 = 8$$

  - **Kết luận:** Thời gian giải thuật toán dự kiến sẽ lâu hơn **8 lần**.
- **b. Hệ số tăng kích thước n với máy tính mới nhanh gấp 1000 lần**
  - Gọi tốc độ giải quyết của máy tính cũ là S, tốc độ của máy mới là $1000 \cdot S$.
  - Giả sử kích thước bài toán lớn nhất máy cũ giải được trong thời gian t là n_{old}. Ta có phương trình: $t = \frac{\frac{1}{3}n_{old}^3}{S}$.
  - Kích thước bài toán lớn nhất máy mới giải được trong cùng thời gian t là n_{new}. Ta có phương trình: $t = \frac{\frac{1}{3}n_{new}^3}{1000 \cdot S}$.
  - Vì thời gian t là như nhau, ta cho 2 vế bằng nhau:

$$\frac{\frac{1}{3}n_{old}^3}{S} = \frac{\frac{1}{3}n_{new}^3}{1000 \cdot S} \implies n_{old}^3 = \frac{n_{new}^3}{1000} \implies n_{new}^3 = 1000 \cdot n_{old}^3$$

$$\implies n_{new} = \sqrt[3]{1000} \cdot n_{old} = 10 \cdot n_{old}$$

- **Kết luận:** Với chiếc máy tính nhanh gấp 1000 lần, hệ số gia tăng kích thước bài toán (factor) trong cùng một khoảng thời gian là **10**. Tức là máy mới giải quyết được hệ phương trình lớn gấp 10 lần máy cũ.

Bài 8

- **Giải thích chung:** Đề bài yêu cầu xác định giá trị của hàm thay đổi như thế nào khi đối số n tăng gấp 3 lần (tức là thay n bằng $3n$). Ta sẽ xét tỉ số $\frac{f(3n)}{f(n)}$ hoặc mô tả sự thay đổi của hàm.
- **a.** $f(n) = n \log_2 n$
  - Thay n bằng $3n$, ta có: $f(3n) = 3n \log_2(3n) = 3n(\log_2 n + \log_2 3) = 3n \log_2 n + 3n \log_2 3$.
  - **Kết luận:** Giá trị tăng lên gấp $3\frac{\log_2(3n)}{\log_2 n}$ lần.
- **b.** $f(n) = n$
  - Thay n bằng $3n$, ta có $f(3n) = 3n$.
  - **Kết luận:** Giá trị tăng lên **gấp 3 lần**.
- **c.** $f(n) = n^3$
  - Thay n bằng $3n$, ta có $f(3n) = (3n)^3 = 27n^3$.
  - **Kết luận:** Giá trị tăng lên **gấp 27 lần**.
- **d.** $f(n) = n^2$
  - Thay n bằng $3n$, ta có $f(3n) = (3n)^2 = 9n^2$.
  - **Kết luận:** Giá trị tăng lên **gấp 9 lần**.
- **e.** $f(n) = n!$
  - Thay n bằng $3n$, ta có $f(3n) = (3n)!$.
  - Tỉ số: $\frac{(3n)!}{n!} = (n+1)(n+2)\dots(3n)$.
  - **Kết luận:** Giá trị tăng lên **gấp** $(n+1)(n+2)\dots(3n)$ **lần**.
- **f.** $f(n) = 2^n$
  - Thay n bằng $3n$, ta có $f(3n) = 2^{3n} = (2^n)^3$.

- **Kết luận:** Giá trị mới sẽ bằng **lập phương** của giá trị ban đầu (hay tăng gấp 2^{2n} lần).

Bài 9

- **Giải thích chung:** So sánh bậc phát triển (order of growth) của hàm thứ nhất so với hàm thứ hai: thấp hơn (lower), bằng nhau (same), hoặc cao hơn (higher) khi xét trong giới hạn Big-O/Θ.
- **a.** $n(n+1)$ **và** $2000n^2$
  - $n(n+1) = n^2 + n \approx \Theta(n^2)$. Hàm thứ hai cũng là $\Theta(n^2)$.
  - **Kết luận: Bằng nhau (Same)**.
- **b.** $100n^2$ **và** $0.01n^3$
  - Bậc của n^2 luôn nhỏ hơn bậc của n^3 khi n tiến tới vô cùng, bất kể hệ số.
  - **Kết luận: Thấp hơn (Lower)**.
- **c.** $\log_2 n$ **và** $\ln n$
  - Theo công thức đổi cơ số logarit: $\ln n = \frac{\log_2 n}{\log_2 e}$. Hai hàm chỉ lệch nhau một hằng số.
  - **Kết luận: Bằng nhau (Same)**.
- **d.** $\log_2^2 n$ **và** $\log_2 n^2$
  - Viết lại: $\log_2^2 n = (\log_2 n)^2$, trong khi $\log_2 n^2 = 2\log_2 n$. Rõ ràng bậc của $(\log_2 n)^2$ phát triển nhanh hơn $2\log_2 n$.
  - **Kết luận: Cao hơn (Higher)**.
- **e.** 2^{n-1} **và** 2^n
  - Viết lại: $2^{n-1} = \frac{1}{2} \cdot 2^n$. Hai hàm này chỉ khác biệt nhau bởi một hằng số nhân là $\frac{1}{2}$.
  - **Kết luận: Bằng nhau (Same)**.
- **f.** $(n-1)!$ **và** $n!$
  - Viết lại: $n! = n \cdot (n-1)!$. Hàm $n!$ lớn hơn $(n-1)!$ gấp n lần (không phải hằng số). Do đó $(n-1)!$ phát triển chậm hơn nhiều.

– **Kết luận: Thấp hơn (Lower)**.

Bài 10

- **a. Thời gian đếm số hạt thóc theo luật nhân đôi**
  - Số hạt trên ô thứ 1 là 2^0, ô thứ 2 là 2^1, ô thứ 3 là 2^2,..., ô thứ 64 là 2^{63}.
  - Tổng số hạt thóc trên 64 ô cờ là tổng của một cấp số nhân:

$$S = \sum_{i=0}^{63} 2^i = 2^{64} - 1 \approx 1.84 \times 10^{19} \text{ (hạt)}$$

  - Vì mất 1 giây để đếm 1 hạt, tổng thời gian đếm là $2^{64} - 1$ giây. Quy đổi ra năm (giả định 1 năm có $\approx 31,536,000$ giây):

$$t \approx \frac{1.84 \times 10^{19}}{31536000} \approx 5.8 \times 10^{11} \text{ (năm)}$$

  - **Kết luận:** Mất khoảng **584 tỷ năm** để đếm hết số thóc này.

- **b. Thời gian đếm nếu mỗi ô tiếp theo cộng thêm 2 hạt thay vì nhân đôi**
  - Thay vì cấp số nhân, ta có một cấp số cộng. Ô thứ 1 có 1 hạt, ô thứ 2 có $1 + 2 = 3$ hạt, ô thứ 3 có $3 + 2 = 5$ hạt...
  - Số lượng hạt trên ô thứ i là $2i - 1$ (các số lẻ).
  - Tổng số hạt trên 64 ô cờ là tổng của 64 số lẻ đầu tiên:

$$S' = \sum_{i=1}^{64} (2i - 1) = 64^2 = 4096 \text{ (hạt)}$$

  - Thời gian đếm là 4096 giây. Quy đổi ra giờ, phút, giây:

$$4096 \text{ giây} = 68 \text{ phút } 16 \text{ giây} = 1 \text{ giờ } 8 \text{ phút } 16 \text{ giây}$$

  - **Kết luận:** Mất khoảng hơn **1 tiếng đồng hồ** để đếm hết số thóc.

Exercises 2.2

Bài 1

- **Giải thích chung:** Biểu thức thời gian thực thi được cho là $f(x) = 50x + 7$. Đây là một hàm tuyến tính (bậc nhất) theo x. Khác với các thuật toán có cấu trúc rẽ nhánh (mà thời gian chạy phụ thuộc vào tình trạng cụ thể của dữ liệu đầu vào như mảng đã sắp xếp hay chưa), biểu thức này luôn trả về một kết quả cố định với cùng một giá trị x. Do đó, không có sự khác biệt giữa các trường hợp tốt nhất, xấu nhất hay trung bình. Ký hiệu phù hợp và chặt chẽ nhất để đánh giá trong mọi trường hợp là Θ.
- **a. Trong trường hợp xấu nhất (in the worst case):**
  - Thời gian thực thi là:
  $$\Theta(x)$$
- **b. Trong trường hợp tốt nhất (in the best case):**
  - Thời gian thực thi là:
  $$\Theta(x)$$
- **c. Trong trường hợp trung bình (in the average case):**
  - Thời gian thực thi là:
  $$\Theta(x)$$

Bài 4

- **a. Bảng 2.1 có chứng minh được thứ tự tăng trưởng (order of growth) bằng toán học không?**
  - **Trả lời:** Không. Các giá trị trong bảng chỉ là những ví dụ thực nghiệm cho một số giá trị n hữu hạn. Trong phân tích thuật toán, ta đánh giá dựa trên hành vi tiệm cận (asymptotic behavior) khi n tiến tới vô cực ($n \to \infty$). Dữ liệu thực nghiệm không thể dùng làm bằng chứng toán học tuyệt đối.
- **b. Chứng minh các hàm được liệt kê theo thứ tự tăng trưởng tăng dần**
  - Ta cần chứng minh với mỗi cặp hàm liền kề $f(n)$ và $g(n)$, ta có $f(n) \in O(g(n))$ nhưng $f(n) \notin \Theta(g(n))$, hay nói cách khác $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$.

- $\lim_{n\to\infty} \frac{\log n}{n} = 0$ (Hàm logarit tăng chậm hơn hàm đa thức).
- $\lim_{n\to\infty} \frac{n}{n\log_2 n} = \lim_{n\to\infty} \frac{1}{\log_2 n} = 0$.
- $\lim_{n\to\infty} \frac{n\log_2 n}{n^2} = \lim_{n\to\infty} \frac{\log_2 n}{n} = 0$.
- $\lim_{n\to\infty} \frac{n^2}{n^3} = \lim_{n\to\infty} \frac{1}{n} = 0$.
- $\lim_{n\to\infty} \frac{n^3}{2^n} = 0$ (Hàm đa thức tăng chậm hơn hàm mũ).
- $\lim_{n\to\infty} \frac{2^n}{n!} = 0$ (Hàm mũ tăng chậm hơn hàm giai thừa).
- **Kết luận:** Dãy hàm thực sự được sắp xếp theo mức độ tăng trưởng tăng dần.

Bài 5

Sắp xếp các hàm theo thứ tự tăng trưởng từ thấp đến cao:

- **Phân tích từng hàm (bỏ qua hằng số):**
  - $2\lg(n+50)^5 = 10\lg(n+50) \in \Theta(\log n)$
  - $\ln^3 n = (\ln n)^3 \in \Theta(\log^3 n)$
  - $\sqrt{n} = n^{0.5} \in \Theta(n^{0.5})$
  - $0.05n^{10} + 3n^3 + 1 \in \Theta(n^{10})$
  - $3^{2n} = (3^2)^n = 9^n \in \Theta(9^n)$
  - $3^{3n} = (3^3)^n = 27^n \in \Theta(27^n)$
  - $(n^2+3)! \in \Theta((n^2)!)$ (Hàm giai thừa bậc cao, tăng cực nhanh)
- **Thứ tự kết quả:**

  $2\lg(n+50)^5 \prec \ln^3 n \prec \sqrt{n} \prec 0.05n^{10} + 3n^3 + 1 \prec 3^{2n} \prec 3^{3n} \prec (n^2+3)!$

Bài 6

- **a. Chứng minh đa thức bậc k, $p(n) = a_k n^k + a_{k-1}n^{k-1} + \cdots + a_0$ với $a_k > 0$, thuộc $\Theta(n^k)$.**
  - Ta sử dụng phương pháp giới hạn:

$$\lim_{n\to\infty} \frac{p(n)}{n^k} = \lim_{n\to\infty} \frac{a_k n^k + a_{k-1}n^{k-1} + \cdots + a_0}{n^k}$$
$$= \lim_{n\to\infty} \left(a_k + \frac{a_{k-1}}{n} + \cdots + \frac{a_0}{n^k}\right) = a_k$$

- Vì $a_k > 0$ là một hằng số dương, theo định lý về giới hạn trong phân tích Big-O, ta có $p(n) \in \Theta(n^k)$.

- **b. Chứng minh các hàm mũ a^n có bậc tăng trưởng khác nhau với các giá trị cơ số $a > 0$ khác nhau.**
  - Giả sử có 2 cơ số a và b sao cho $0 < a < b$. Ta xét giới hạn tỉ số của chúng:
  $$\lim_{n\to\infty} \frac{a^n}{b^n} = \lim_{n\to\infty} \left(\frac{a}{b}\right)^n$$
  - Vì $a < b$ nên $0 < \frac{a}{b} < 1$. Do đó giới hạn trên bằng 0.
  - Điều này chứng tỏ $a^n \in O(b^n)$ nhưng $a^n \notin \Theta(b^n)$. Vậy các hàm mũ với cơ số khác nhau sẽ có bậc tăng trưởng khác nhau.

Bài 7

- **a. Nếu $t(n) \in O(g(n))$, thì $g(n) \in \Omega(t(n))$.**
  - **Đúng.** Theo định nghĩa, $t(n) \in O(g(n)) \implies t(n) \leq c \cdot g(n)$ với mọi $n \geq n_0$ (trong đó $c > 0$).
  - Chia 2 vế cho c (vì $c > 0$), ta được $g(n) \geq \frac{1}{c}t(n)$. Gọi $c' = \frac{1}{c} > 0$, ta có $g(n) \geq c' \cdot t(n)$ với mọi $n \geq n_0$. Đúng với định nghĩa của Ω.

- **b. $\Theta(\alpha g(n)) = \Theta(g(n))$, với $\alpha > 0$.**
  - **Đúng.** Hằng số nhân không làm thay đổi bậc Θ. Bất kỳ hàm $f(n)$ nào bị chặn bởi $c_1\alpha g(n) \leq f(n) \leq c_2\alpha g(n)$ cũng có thể viết lại thành $c_1' g(n) \leq f(n) \leq c_2' g(n)$ (bằng cách đặt hằng số mới), thỏa mãn định nghĩa $\Theta(g(n))$.

- **c. $\Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$.**
  - **Đúng.** Đây chính xác là định nghĩa chính thức của ký hiệu Θ. Một hàm thuộc $\Theta(g(n))$ khi và chỉ khi nó vừa bị chặn trên (O), vừa bị chặn dưới (Ω) bởi $g(n)$.

- **d. Với mọi hàm không âm $t(n)$ và $g(n)$, hoặc $t(n) \in O(g(n))$, hoặc $t(n) \in \Omega(g(n))$, hoặc cả hai.**
  - **Sai (Cần đưa phản ví dụ).** Chọn $t(n)$ và $g(n)$ như sau:
  $$t(n) = \begin{cases} n, & \text{nếu } n \text{ chẵn} \\ n^2, & \text{nếu } n \text{ lẻ} \end{cases}$$

$$g(n) = \begin{cases} n^2, & \text{nếu } n \text{ chẵn} \\ n, & \text{nếu } n \text{ lẻ} \end{cases}$$

- Không thể tìm được hằng số c để $t(n) \leq c \cdot g(n)$ đúng với mọi n đủ lớn (vì khi n lẻ, n^2 sẽ vượt qua $c \cdot n$). Tương tự với chiều ngược lại. Do đó không thỏa mãn điều kiện nào.

Bài 8

- **Giải thích chung:** Định lý của phần này phát biểu cho Big-O là: Nếu $t_1(n) \in O(g_1(n))$ và $t_2(n) \in O(g_2(n))$, thì $t_1(n)+t_2(n) \in O(\max(g_1(n), g_2(n)))$. Đề bài yêu cầu chứng minh định lý tương tự cho Ω và Θ.

- **a. Chứng minh cho ký hiệu Ω**
  - Giả thiết: $t_1(n) \in \Omega(g_1(n))$ và $t_2(n) \in \Omega(g_2(n))$.
  - Theo định nghĩa, tồn tại hằng số $c_1, c_2 > 0$ và $n_1, n_2 \geq 0$ sao cho:

$$t_1(n) \geq c_1 g_1(n) \text{ với mọi } n \geq n_1$$

$$t_2(n) \geq c_2 g_2(n) \text{ với mọi } n \geq n_2$$

  - Xét với mọi $n \geq \max(n_1, n_2)$. Ta có:

$$t_1(n) + t_2(n) \geq c_1 g_1(n) + c_2 g_2(n)$$

  - Đặt $c = \min(c_1, c_2) > 0$. Ta có:

$$t_1(n) + t_2(n) \geq c(g_1(n) + g_2(n))$$

  - Vì $g_1(n), g_2(n) \geq 0$, nên $g_1(n)+g_2(n) \geq \max(g_1(n), g_2(n))$. Do đó:

$$t_1(n) + t_2(n) \geq c \cdot \max(g_1(n), g_2(n))$$

  - Theo định nghĩa, điều này chứng tỏ $t_1(n)+t_2(n) \in \Omega(\max(g_1(n), g_2(n)))$ (ĐPCM).

- **b. Chứng minh cho ký hiệu Θ**
  - Giả thiết: $t_1(n) \in \Theta(g_1(n))$ và $t_2(n) \in \Theta(g_2(n))$.
  - Từ định nghĩa $\Theta = O \cap \Omega$, ta suy ra:
    1. $t_1(n) \in O(g_1(n))$ và $t_2(n) \in O(g_2(n)) \implies t_1(n) + t_2(n) \in O(\max(g_1(n), g_2(n)))$ (theo định lý gốc).

2. $t_1(n) \in \Omega(g_1(n))$ và $t_2(n) \in \Omega(g_2(n)) \implies t_1(n) + t_2(n) \in \Omega(\max(g_1(n), g_2(n)))$ (vừa chứng minh ở câu a).

– Vì tổng $t_1 + t_2$ đồng thời thỏa mãn cả hai chiều chặn trên (O) và chặn dưới (Ω), ta có kết luận:

$$t_1(n) + t_2(n) \in \Theta(\max(g_1(n), g_2(n))) \text{ (ĐPCM).}$$

Exercise 3: Selection Sort

- **1. Pseudocode (Mã giả) cho thuật toán Selection Sort:**

```
Algorithm SelectionSort(A[1..n])
// Sắp xếp mảng A theo thứ tự tăng dần bằng Selection Sort
// Input: Mảng A[1..n] gồm n phần tử có thể so sánh được
// Output: Mảng A[1..n] đã được sắp xếp tăng dần
for i <- 1 to n - 1 do
    min_idx <- i
    for j <- i + 1 to n do
        if A[j] < A[min_idx]
            min_idx <- j
    swap A[i] and A[min_idx]
```

- **2. Tại sao chỉ cần thực hiện quy trình cho $n-1$ phần tử đầu tiên thay vì toàn bộ n phần tử?**

  – **Giải thích:** Sau khi thực hiện xong $n-1$ bước lặp (tức là i chạy từ 1 đến $n-1$), chúng ta đã tìm và đặt đúng vị trí cho $n-1$ phần tử nhỏ nhất của mảng vào các vị trí từ $A[1]$ đến $A[n-1]$.

  – Phần tử duy nhất còn sót lại ở vị trí $A[n]$ lúc này bắt buộc phải là phần tử lớn nhất trong mảng. Do đó, nó đã tự động nằm đúng ở vị trí cuối cùng mà không cần thêm bất kỳ vòng lặp hay thao tác hoán vị nào nữa.

- **3. Đánh giá thời gian chạy (Running time) bằng Big-O:**

  – Phép toán cơ bản của thuật toán là phép so sánh ở lệnh `A[j] < A[min_idx]`.

- Tổng số phép so sánh cho một mảng kích thước n được tính bằng:

$$C(n) = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} 1 = \sum_{i=1}^{n-1} (n-i) = \frac{n(n-1)}{2}$$

- **Best case (Trường hợp tốt nhất):** Kể cả khi mảng đã được sắp xếp sẵn từ trước, thuật toán vẫn phải dùng hai vòng lặp lồng nhau để quét qua toàn bộ phần còn lại của mảng nhằm tìm giá trị nhỏ nhất. Số phép so sánh không hề thay đổi. Do đó độ phức tạp là $O(n^2)$.
- **Worst case (Trường hợp xấu nhất):** Tương tự, dù mảng bị sắp xếp ngược, số phép so sánh vẫn giữ nguyên là $\frac{n(n-1)}{2}$. Độ phức tạp vẫn là $O(n^2)$.
- **Kết luận:** $T(n) \in O(n^2)$ cho mọi trường hợp (best, worst, average).

- **4. (*) Chứng minh tính đúng đắn bằng Loop Invariant (Bất biến vòng lặp):**

  - **Phát biểu Loop Invariant:** Ở thời điểm bắt đầu mỗi vòng lặp `for` ngoài cùng (với biến lặp i), mảng con $A[1..i-1]$ chứa $i-1$ phần tử nhỏ nhất của mảng ban đầu và đã được sắp xếp tăng dần. Đồng thời, mọi phần tử nằm trong mảng con chưa sắp xếp $A[i..n]$ đều lớn hơn hoặc bằng mọi phần tử trong $A[1..i-1]$.
  - **Initialization (Khởi tạo):** Trước vòng lặp đầu tiên ($i = 1$), mảng con $A[1..0]$ là một mảng rỗng. Mảng rỗng hiển nhiên thỏa mãn tính chất "đã được sắp xếp"và "chứa 0 phần tử nhỏ nhất". Bất biến hoàn toàn đúng lúc khởi tạo.
  - **Maintenance (Duy trì):** Giả sử bất biến đúng tại đầu bước i. Trong vòng lặp này, ta tìm phần tử nhỏ nhất trong $A[i..n]$ và hoán vị nó với $A[i]$. Vì $A[i]$ giờ đây là phần tử nhỏ nhất của phần chưa sắp xếp $A[i..n]$, và theo giả thiết mọi phần tử của $A[i..n]$ đều $\geq$ phần tử lớn nhất trong $A[1..i-1]$, nên mảng con $A[1..i]$ hiện tại sẽ chứa i phần tử nhỏ nhất và tiếp tục được sắp xếp hợp lệ. Bất biến được duy trì trước khi bước sang vòng lặp $i+1$.
  - **Termination (Kết thúc):** Vòng lặp kết thúc khi $i = n$. Theo tính chất bất biến, lúc này mảng con $A[1..n-1]$ đã được sắp xếp và chứa $n-1$ phần tử nhỏ nhất. Phần tử còn lại ở $A[n]$ bắt buộc phải là phần tử lớn nhất. Vậy toàn bộ mảng $A[1..n]$ đã được sắp xếp đúng. Thuật toán hoàn toàn chính xác.

- **5. Triển khai thuật toán và đo thời gian (Mã Python minh họa):** [language=Python] import time import random import matplotlib.pyplot as plt

  def selection$_sort(arr) : n = len(arr) for i in range(n-1) : min_idx = i for j in range(i+1, n) : if arr[j] < arr[min_idx] : min_idx = j arr[i], arr[min_idx] = arr[min_idx], arr[i]$

Test voi cac kich thuoc mang khac nhau sizes = [100, 500, 1000, 2000, 4000]
times = []

for n in sizes: arr = [random.randint(0, 10000) for $_i n range(n)] start = time.time() selection_sort(ar$
$time.time() times.append(end - start)$

Ve bieu do plt.plot(sizes, times, marker='o') plt.xlabel('Kich thuoc dau vao (n)') plt.ylabel('Thoi gian chay (giay)') plt.title('Do phuc tap thoi gian cua Selection Sort') plt.grid(True) plt.show() *Lưu ý: Bạn copy code trên chạy ở máy cá nhân (VSCode/PyCharm) để lấy ảnh biểu đồ, sau đó dùng lệnh* `\includegraphics` *chèn ảnh đó vào Overleaf nhé.*